Lightswarm

In this project, three ESPs work as a self-organizing swarm to send light data to a server (Raspberry Pi). The ESP that detects the brightest light with its photoresistor becomes the master, sending its data to the server; all ESPs in the swarm multicast their data to the rest of the swarm, so that each can internally keep track of the state of the swarm and who is the master.

Schematics

Raspberry Pi Schematic

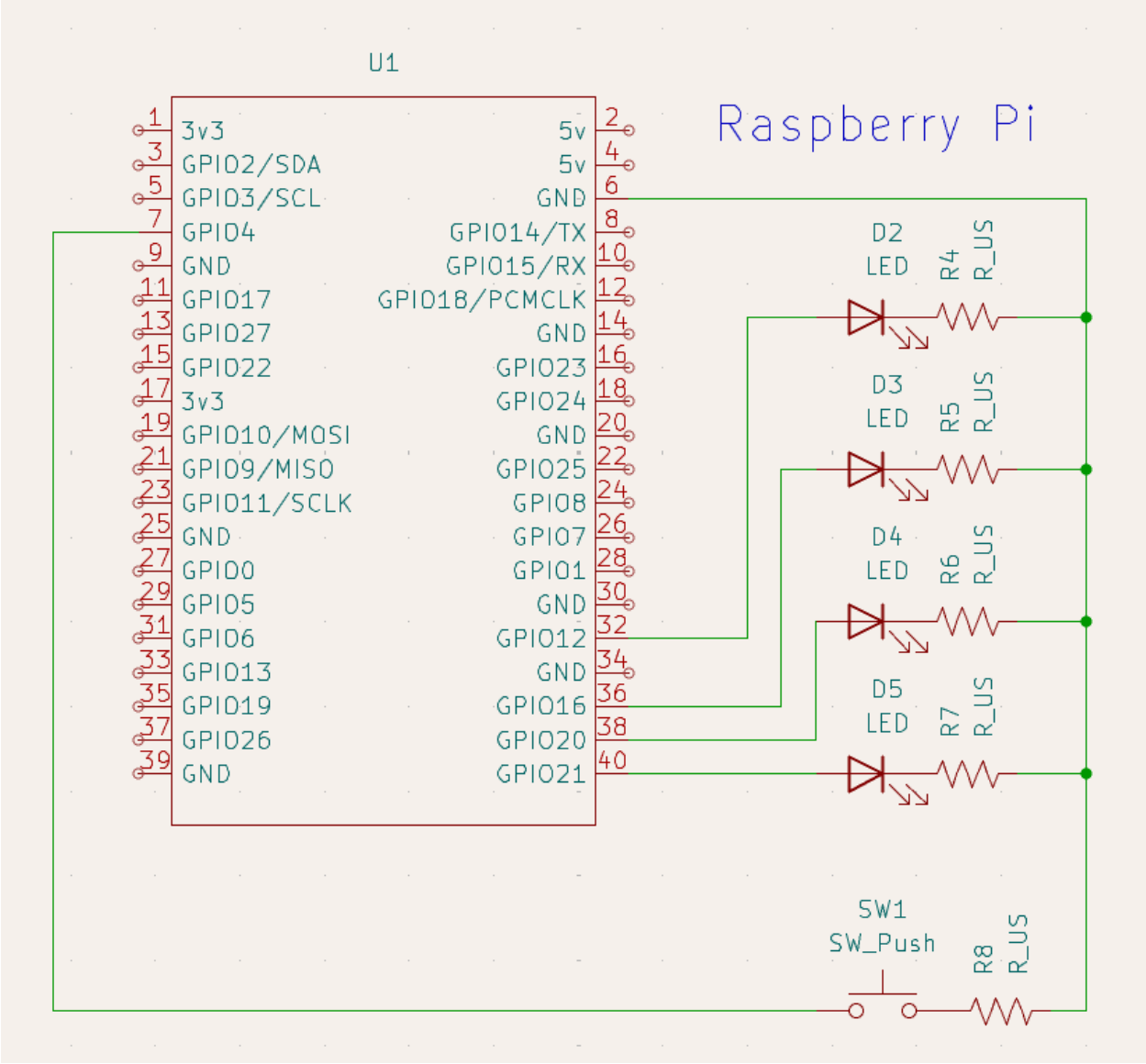


Figure 1: A schematic of the Raspberry Pi circuit.

ESP Schematic

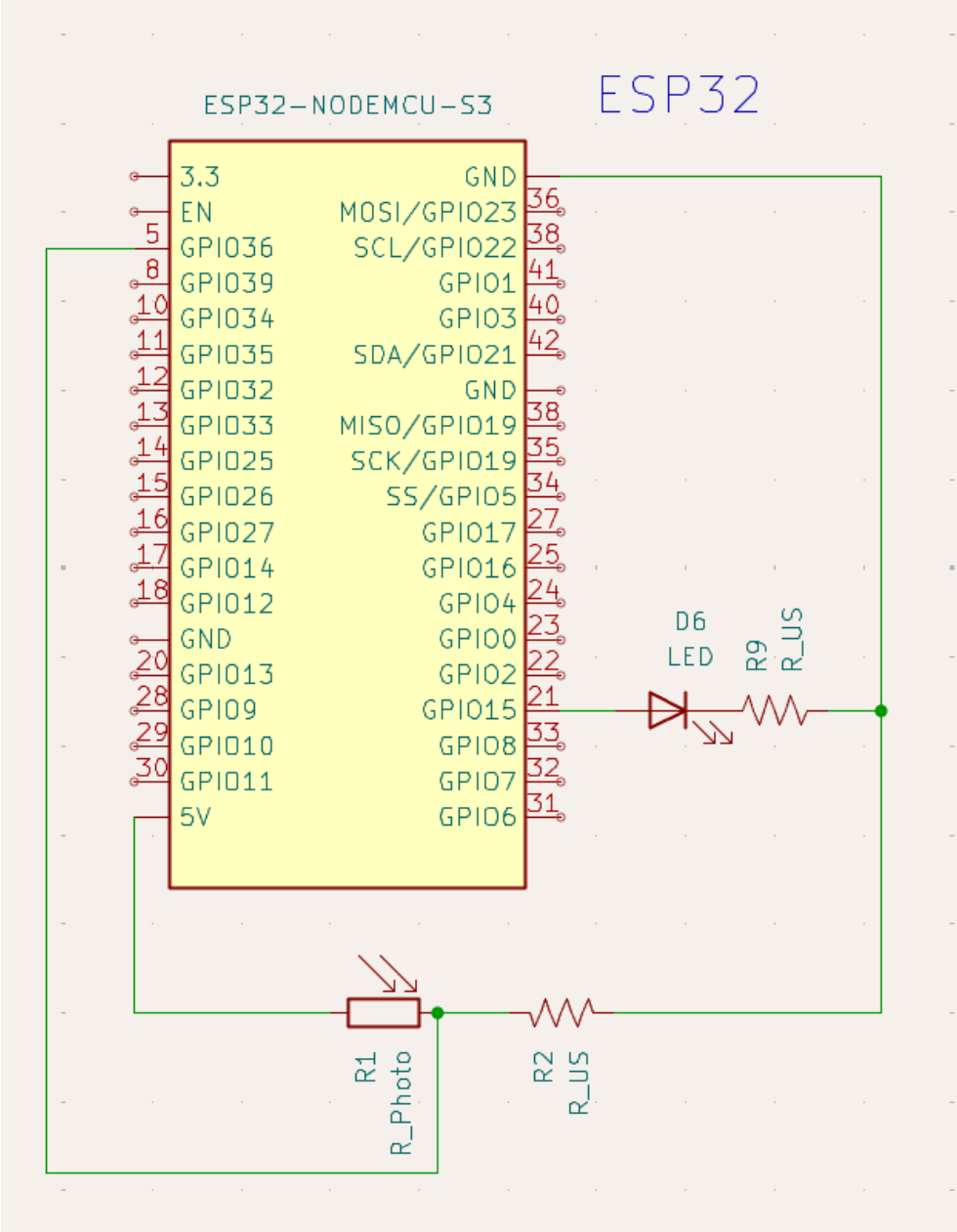


Figure 2: A schematic of the ESP circuit. The swarm includes three ESPs with an identical layout.

Communication Protocol

The lightswarm uses UDP for communication. There are four main types of packets sent by the devices. The first is from the swarm ESPs to each other, containing their latest photoresistor readings. The second is from the master ESP to the server, containing all swarm information.

The third is from the server to all ESPs, telling them to reset. The fourth introduces the server to the ESPs so they know which IP to send server log packets to.

I made a change to the provided implementation where the swarm uses one port (for sending light update packets to each other, or receiving messages from the server) and the server uses a separate port to receive server log information. This is because under the provided implementation, the server was receiving and reacting to all light update packets, rather than just the summary sent by the master, which seemed to defeat the purpose of using the master/slave configuration for the swarm.

Light Update Packet

These UDP packets are sent between the ESPs. These are sent to the swarm's internal port. They are 8 bytes long and contain the following information:

Byte Number	Data
0	Start byte
1	Packet type (0)
2	Final section of this ESP's IP address
3	Master state (0 = slave, 1 = master)
4	Software version number
5	Photoresistor reading (high byte)
6	Photoresistor reading (low byte)
7	End byte

Server Log Packet

These UDP packets are sent from the ESP currently designated master to the server. These are sent to the server's port. They are of variable length, but contain the following six pieces of information for each swarm member:

Index	Data
0	Swarm ID
1	Master state (0 = slave, 1 = master)
2	Software version number
3	Photoresistor reading

4	Swarm member state (e.g. TO for Too Old or PR for Present)
5	Final section of this ESP's IP address

Each of these data points is delimited by a comma; each swarm member's data is delimited by a pipe (|).

Swarm Reset Message

These UDP packets are broadcast from the server to the ESPs in the event that the server's button is pressed. These are sent to the swarm's port. These packets include a start byte, the byte that indicates the RESET_SWARM command, and an end byte.

Server Logger Message

These UDP packets are broadcast from the server to the ESPs when the server is first booted up, and every 300s after that. These are sent to the swarm's port. These packets include a start byte, the byte that indicates the DEFINE_SERVER_LOGGER command, the four sections of the server's IP address, and an end byte.

Raspberry Pi WiFi Setup and Packet Delivery

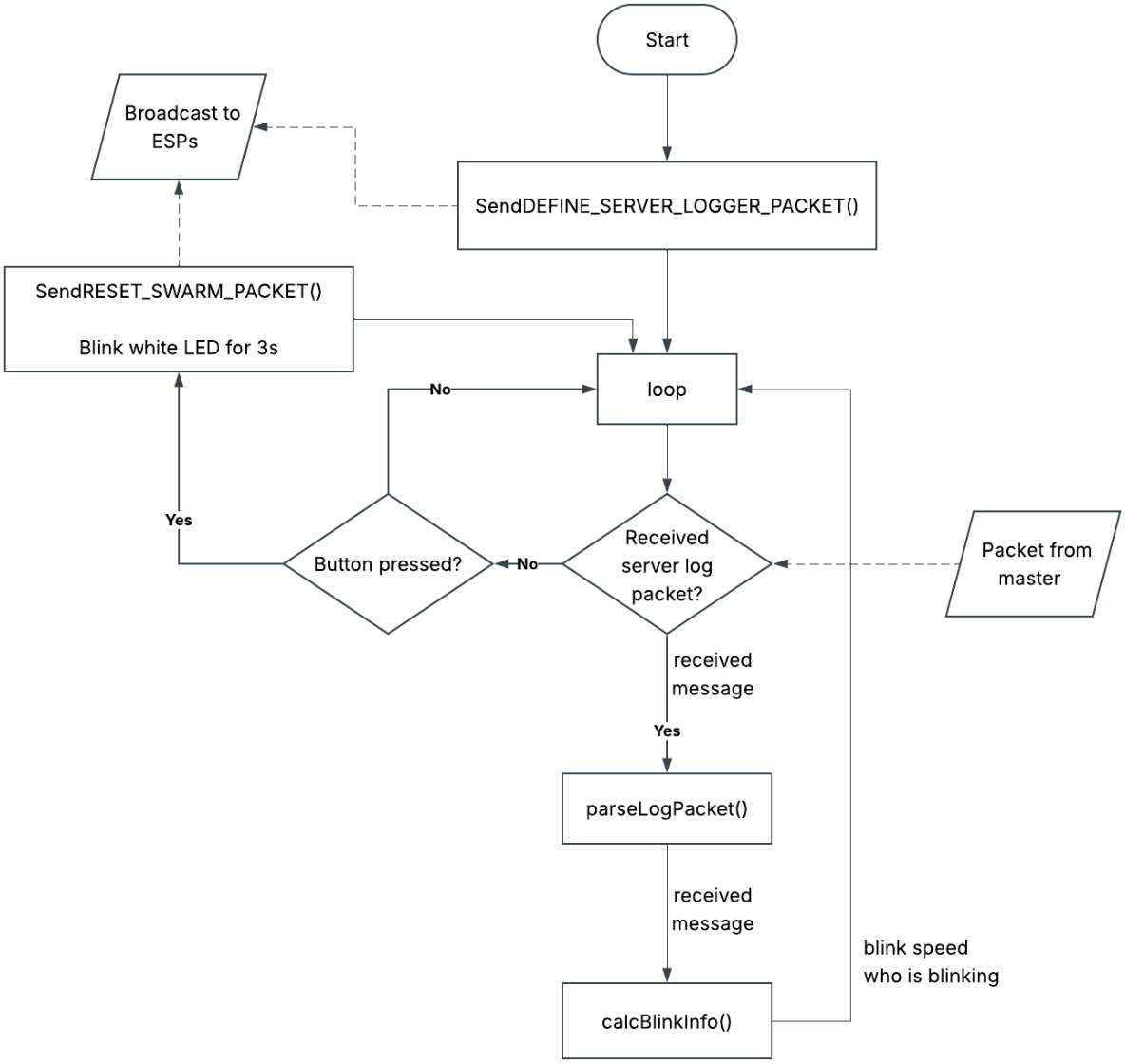


Figure 3: Flow chart of Raspberry Pi functionality

Raspberry Pi IPO

SendDEFINE_SERVER_LOGGER_PACKET:
Input: n/a
Process: Broadcasts own existence so that the swarm identifies the server's IP address
Output: UDP sent to ESP

SendRESET_SWARM_PACKET:
Input: n/a

Process: Broadcasts a command to the swarm to reset
Output: UDP sent to ESP

Main loop:
Input: Log packet from master. Also requires initialization with server information (socket setup and ports for server and swarm)
Process: Attempts to receive and parse log information from master swarm member. Sends an occasional pulse to ESPs identifying itself
Output: If it receives log information, prints that information to the console, and blinks the RGB LED associated with the current master at a speed determined by the average value of the lightswarm.

parseLogPacket:
Input: Server log message received
Process: Friendly prints log message
Output: Printout

calcBlinkInfo:
Input: Server log message received, size of swarm
Process: Finds the average of the photoresistor data from the swarm, and calculates a blink speed ranging from 50 (the fastest, representing an average photoresistor value of 4096) to 2000 (the slowest, representing an average photoresistor value of 0).
Output: Index of master within swarm, speed at which to blink

ESP WiFi Setup and Packet Delivery

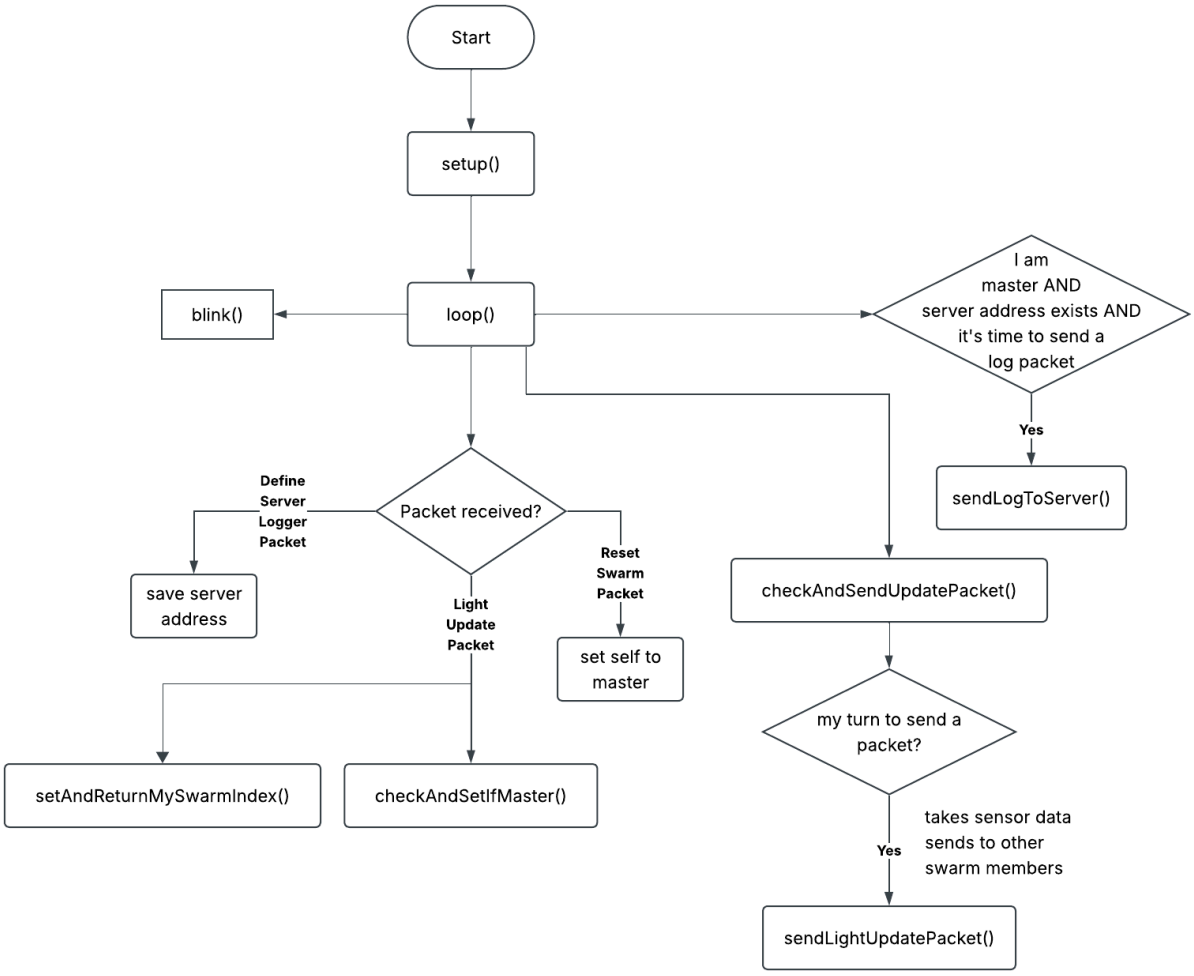


Figure 4: Flow chart of ESP functionality

ESP IPO

setup():
Input: Configuration information - version number, network SSID and password, port number for server, port number for swarm
Process: Initial setup. Creates and initializes all values to be used by this swarm member. Sets a lot of default values e.g. for swarmAddresses to be overwritten later.
Output: Prints monitoring information.

loop():
Input: Data from the sensor, and UDP packets from the rest of the swarm.
Process: Gathers data from the light sensor and processes it to be sent in a packet if it's currently time to do so. If a UDP packet has been received, parses that information.

Output: Prints monitoring information; calls a variety of other functions.

setAndReturnMySwarmIndex():
Input: Address provided in a UDP packet from another swarm member.
Process: Checks whether this provided address matches one of three conditions: a) is from a swarm member we already know about; b) is a fresh, expected connection; or c) is a fresh, unexpected connection. If case C, overwrite the oldest connection (the one it's been the longest since we received data from).
Output: Updates known swarm addresses array to include the new information, or, if nothing is changed, simply returns the known index.

checkAndSetIfMaster():
Input: Light information from packets received from other swarm members.
Process: Compares this swarm member's light sensor data against all other swarm members. If we're sensing the brightest light, we become the master.
Output: Whether or not this swarm member is currently the master.

sendLightUpdatePacket():
Input: Current light sensor data, and swarm member network information.
Process: Builds and multicasts a new UDP packet containing the meta-information about this swarm member, and the current photoresistor data.
Output: UDP packet broadcast to the rest of the swarm.

broadcastARandomUpdatePacket():
Input: Network information.
Process: Determines whether the conditions are met to send a light update packet. The conditions are met if it's been at least 200ms since the last packet was received and, of all of the swarm members, it's been the longest since the current ESP has sent a message OR if it's been at least 210 ms since the last packet was received, even if it isn't this one's "turn".
Output: n/a

sendLogToServer():
Input: Network information, information on the current state of the swarm.
Process: If this ESP is the master, and it's been at least 200ms since the server sent data, sends a UDP packet to the RPi to update it on the current state of the swarm.
Output: UDP packet to the server.

blink():
Input: Latest photoresistor reading.
Process: Calculates new blink speed based on photoresistor reading and determines (based on clock and blink speed) whether to turn the builtin LED on or off.
Output: Change to LED